Assignment: Real-World Serverless ETL with CI/CD

Goal: Build a real-world ETL pipeline where third-party data is stored in Amazon S3, processed by AWS Lambda, loaded into Amazon DynamoDB, version-controlled in GitHub, and validated through GitHub Actions and AWS CodePipeline.

Submit: GitHub repository link + required screenshots listed in this brief.

1. Scenario

Choose one real-world third-party dataset and create a small data engineering use case around it. Examples: weather or air quality, earthquake/public safety, open government data, transit data, stock/crypto data, or product/e-commerce API data.

Dataset type	Possible ETL scenario
Weather / air quality	Clean city readings and store latest metrics
Earthquake / public safety	Load validated events for alerting or analytics
Product / e-commerce API	Store cleaned catalog or order records
Open government / transit	Prepare records for searchable dashboard data

2. Required Architecture

```
Third-party data source
-> raw file
-> Amazon S3 raw/ prefix
-> AWS Lambda ETL function
-> Amazon DynamoDB clean table
-> audit summary / CloudWatch logs
```

Lambda may be triggered by an S3 event or by receiving a pre-signed S3 URL. Keep the project small: one dataset, one S3 bucket, one Lambda function, one DynamoDB table.

3. Minimum AWS and DevOps Resources

1. S3 bucket with raw/ prefix
2. DynamoDB table such as clean_records
3. Lambda function for ETL processing
4. IAM role for Lambda using least privilege
5. GitHub repository
6. GitHub Actions workflow
7. AWS CodePipeline with Source and Build stages

4. ETL Requirements

Stage	Requirement
Extract	Read raw data from S3 or a pre-signed S3 URL.
Transform	Remove invalid records, standardize at least two fields, and add one derived field.
Load	Write clean records to DynamoDB using a meaningful partition key.
Audit	Log total input records, inserted records, rejected records, and timestamp.

Suggested DynamoDB design: Table **clean_records**, partition key **record_id** as String, capacity mode **On-demand**.

5. GitHub Repository Requirements

Your repository should be clean, readable, and organized like a mini production project.

```
etl-s3-lambda-dynamodb/
README.md
lambda_function.py
requirements.txt
buildspec.yml
sample_data/sample_raw_data.csv
screenshots/
.github/workflows/ci.yml
```

README.md must include: project title, dataset source, scenario, architecture diagram, AWS services used, ETL rules, DynamoDB table design, testing steps, GitHub Actions summary, and AWS CodePipeline summary.

6. GitHub Actions Requirement

Create a workflow that runs on push or pull request. It should check out code, set up Python, install dependencies, and validate Lambda syntax. Submit a screenshot of a successful run.

```
name: CI
on: [push, pull_request]
jobs:
validate:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- uses: actions/setup-python@v5
with: { python-version: "3.11" }
- run: pip install -r requirements.txt || true
- run: python -m py_compile lambda_function.py
```

7. AWS CodePipeline Requirement

Create an AWS CodePipeline connected to the GitHub repository. It must include at least a GitHub Source stage and an AWS CodeBuild Build stage. Submit a screenshot of the successful pipeline execution.

```
version: 0.2
phases:
build:
commands:
- pip install -r requirements.txt || true
- python -m py_compile lambda_function.py
- echo "Build completed"
artifacts:
files:
- lambda_function.py
- requirements.txt
```

8. Security Rules

Do not commit AWS access keys, secret keys, .env files, credentials, generated zip files, or large raw datasets.

Recommended .gitignore entries: **.env**, ***.csv**, **__pycache__/**, **package/**, ***.zip**, **.ipynb_checkpoints/**.

9. Submission Checklist

1. GitHub repository link
2. Screenshot of raw file in S3
3. Screenshot of Lambda execution or CloudWatch logs
4. Screenshot of clean records in DynamoDB
5. Screenshot of successful GitHub Actions run
6. Screenshot of successful AWS CodePipeline run
7. Short explanation of the ETL flow

10. Evaluation Rubric

Criteria	Marks
Real-world dataset and scenario	10
S3 raw data setup	10
Lambda ETL logic	20
DynamoDB loading and table design	15
IAM/security awareness	10
GitHub repo structure	10
GitHub Actions workflow	10
AWS CodePipeline screenshot	10
README clarity	5
Total	100

11. Reflection Questions

1. Why did you choose DynamoDB for this project?
2. What is your partition key and why?
3. What transformation rules did Lambda apply?
4. What did GitHub Actions validate?
5. What did AWS CodePipeline do?
6. Which files should never be committed to GitHub and why?

12. Final Expected Outcome

Your final project should show a real-world dataset, a working serverless ETL flow, raw data stored in S3, clean records stored in DynamoDB, a clean GitHub repo, a passing GitHub Actions workflow, and a successful AWS CodePipeline execution.